

LLMKernelBench: Benchmarking Large Language Models on Software Vulnerability Detection in Linux Kernel

Arastoo Zibaeirad , Rodrigo Pato Nogueira , and Marco Vieira 

Abstract—Large Language Models (LLMs) demonstrate strong capabilities in code-related tasks, however their effectiveness in Software Vulnerability Detection (SVD) remains poorly understood due to inadequate evaluation frameworks. Existing benchmarks suffer from training data contamination, isolated function evaluation without cross-component context, lack of strict vulnerable-patched pairing, and binary classification without hierarchy-aware Common Weakness Enumeration (CWE) assessment, which prevents a reliable measurement of security reasoning versus pattern matching on leaked data. We introduce LLMKernelBench, a rigorous benchmark comprising 417 real-world Linux kernel vulnerabilities across 74 CWE types, split into a Primary Benchmark Dataset (PBD; 314 samples, ≤ 2024) and a Leakage Free Dataset (LFD; 103 samples, 2025 post-cutoff), with context-aware evaluation at three granularity levels and hierarchy-aware metrics quantifying semantic proximity in misclassifications. We evaluate seven LLMs spanning code-specialized and general-purpose architectures. Binary vulnerability detection is near-random ($\sim 50\%$ accuracy) and strongly biased: some models label $> 75\%$ of samples as vulnerable, while others mostly label them as non-vulnerable. CWE prediction is effectively unusable, with an average Top-1 accuracy of 1.4% (best: 3.3%) and a 12.3% hierarchy proximity score, providing little reliable exact or taxonomy-level signal. On the leakage-free 2025 split, binary accuracy remains near-random and robustness to multi-file abstraction is model-specific rather than tied to specialization, with code-specialized and general-purpose models degrading by 2.6% and 4.5% on average, respectively. The micro-to-macro CWE-accuracy gap is larger on LFD (6.6 points) than on PBD (0.9 points), which is consistent with sensitivity to class frequency but does not identify an internal model mechanism. Code and data are available at <https://anonymous.4open.science/r/LLMKernelBench-785B/>.

Index Terms—Large Language Models, Software Security, Software Vulnerability Detection

I. INTRODUCTION

Software vulnerabilities pose a critical threat to modern computing infrastructure. The number of identified CVEs surged to over 29,000 in 2023, up from 20,153 in 2021 [1], underscoring the urgent need for automated Software Vulnerability Detection (SVD). Real-world vulnerabilities rarely occur in isolated code fragments, instead involving intricate interactions across functions, files, and modules. Detecting them requires not only identifying vulnerable code but also classifying the specific Common Weakness Enumeration (CWE) category, which is essential for triage, prioritization, and mitigation.

As production systems such as the Linux kernel ($\sim 30\text{M}$ LOC) power Android, servers, and embedded devices, and because diverse, high-impact flaws persist, effective automated detection is critical for software security at scale.

Traditional SVD approaches include Automated Program Repair, fuzzing, and Static Analysis Tools (SATs), but these face key limitations: patch quality issues [2], [3], coverage gaps and logic error blindness [4], [5], and high false positive rates [6]. Recent work has turned to Large Language Models (LLMs) [7]–[11], which demonstrate strong capabilities in code understanding, test generation [12], [13], and various programming tasks [14]. Multiple benchmarks have emerged to evaluate LLM security reasoning, incorporating vulnerability-specific metadata [15]–[17] and CWE-wise analysis [18], [19].

Despite progress, existing benchmarks exhibit critical limitations. First, most rely on historical data that likely overlaps with LLM training corpora, artificially inflating metrics through *training data contamination*, with only limited efforts [19] addressing this via post-cutoff validation. Second, datasets often extract *isolated functions* without surrounding context (macros, type definitions, cross-file dependencies), failing to capture how real vulnerabilities emerge through component interactions. Third, many lack *strict vulnerable-patched pairing*, instead collecting functions from different commits or versions, introducing label noise [20]–[22]. Fourth, evaluations mainly focus on binary classification, overlooking *CWE hierarchy understanding* and whether models capture vulnerability families or simply memorize patterns. These gaps hinder reliable assessment of whether LLMs exhibit genuine security reasoning or pattern matching on contaminated data.

In this paper, we present LLMKernelBench, a rigorous benchmark addressing these limitations through three key design principles. First, we introduce a *temporal split*: the Primary Benchmark Dataset (PBD) covers vulnerabilities through 2024, while the Leakage Free Dataset (LFD) contains only 2025 post-cutoff CVEs, enabling comparison of historical and post-cutoff performance while reducing direct exposure risk in LFD. Second, we adopt a *context-aware* approach, pairing 417 real-world Linux kernel CVEs with strict vulnerable-patched pairing while preserving surrounding non-functional elements that influence vulnerability semantics. Third, we evaluate across three *abstraction levels* (single function, multiple functions, multiple files) to assess context utilization, which is essential for deployment but has not been explored in prior work. We evaluate seven pre-trained LLMs spanning code-specialized (CodeLlama, StarCoder2, Qwen3-Coder) and

Arastoo Zibaeirad, Rodrigo Pato Nogueira, and Marco Vieira are with the University of North Carolina at Charlotte, Charlotte, NC, USA (e-mail: azibaeir@charlotte.edu; rpatodec@charlotte.edu; marco.vieira@charlotte.edu).

general-purpose models (Llama3.1, Mistral, DeepSeek-R1, GPT-4.1-mini) across 74 CWE types, measuring both binary detection and hierarchy-aware CWE classification through novel metrics (Hierarchical Proximity Score (HPS)) that reward taxonomic proximity.

Our work intentionally targets the Linux kernel as a representative example of a *high-assurance, security-critical software system*. The kernel’s extensive use of macros, low-level memory management, and long-evolving coding conventions reflects the complexity of infrastructure software, where vulnerabilities have severe consequences. Accordingly, LLMKernelBench is not designed to characterize LLM performance on general-purpose application code, but rather to stress-test vulnerability-detection capabilities under conditions representative of real-world, safety- and security-critical systems.

Our evaluation addresses four research questions:

- **RQ1: Binary SVD performance:** How effectively can large language models detect whether a code sample is vulnerable or non-vulnerable?
- **RQ2: Consistency across CWEs pillars:** Are model performances consistent across different CWEs pillars?
- **RQ3: Consistency across granularity levels:** Are model performances consistent across different code granularities (file-, function-, and multi-file-level)?
- **RQ4: CWE ranking performance:** When a vulnerability is detected, how accurately can models identify its corresponding CWE category?

Contributions. In summary, this work offers:

- **Leakage-aware benchmark with temporal split.** We introduce a kernel-focused benchmark with explicit temporal contamination control, including 417 real-world Linux kernel CVEs split into PBD (314 samples, ≤ 2024) and LFD (103 samples, 2025 post-cutoff), enabling comparison under substantially reduced direct-exposure risk.
- **Context-aware evaluation at three abstraction levels.** Unlike prior work using isolated functions, we provide strict vulnerable-patched pairing with surrounding context (macros, type definitions, globals) and evaluate at L1 (single function), L2 (multiple functions), and L3 (multiple files) to assess whether models leverage cross-component dependencies.
- **Hierarchy-aware CWE evaluation metric.** We develop the HPS metric combining Top- k accuracy, Mean Reciprocal Rank (MRR), and CWE taxonomy structure to quantify taxonomic proximity in misclassifications, distinguishing nearby hierarchy errors from unrelated labels.
- **Comprehensive evaluation of seven LLMs.** We evaluate code-specialized (CodeLlama, StarCoder2, Qwen3-Coder) and general-purpose models (Llama3.1, Mistral, DeepSeek-R1, GPT-4.1-mini) across 74 CWE types. We focus on open-source models to ensure transparency, reproducibility, and cost-effectiveness.

From a reliability engineering perspective, vulnerability detection models act as *diagnostic components* within a broader software assurance pipeline. Their practical value depends not only on average accuracy, but also on predictable behavior under realistic operating conditions, robustness to unseen inputs, and stable failure modes when confidence is unwarranted.

Our evaluated zero-shot models do not meet these reliability requirements: they exhibit near-random decision accuracy, strong class bias, and substantial between-model variation across abstraction levels. These observed behaviors make them unsuitable as standalone vulnerability detectors in safety- or security-critical systems. By characterizing these failure modes under controlled, leakage-aware conditions, this work provides evidence about the reliability limits of baseline LLM-assisted analysis and a benchmark for assessing future improvements.

The remainder of this paper is structured as follows: §II reviews background and related work. §III presents the benchmark and evaluation protocol. §IV reports findings for RQ1–RQ4. §V discusses implications and limitations. §VI summarizes threats to validity. §VII concludes the paper.

II. BACKGROUND AND RELATED WORK

CWE Taxonomy and Hierarchical Evaluation: The CWE system [23] organizes software weaknesses into a four-level hierarchy: Pillars, Classes, Base weaknesses, and Variants. This enables graduated assessment, where identifying a parent category receives partial taxonomic credit even when the specific variant is misidentified. Our work leverages this hierarchy to measure both exact classification accuracy and taxonomic proximity through the HPS metric.

Vulnerability Datasets – From File-Level to Context-Aware: Early datasets adopted *file-level* labeling, marking entire files touched by security commits as vulnerable [24]–[27]. While enabling large-scale collection, this introduced label noise, class imbalance [20], and difficulty distinguishing security fixes from refactoring [21], [22]. The field shifted toward *function-level* granularity [28], [29], but these often failed to strictly pair vulnerable-patched versions of the *same* function or account for cross-component dependencies.

Unlike prior work extracting isolated functions, LLMKernelBench adopts a *context-aware* approach, pairing line-modified functions with surrounding non-functional elements (macros, type definitions, globals) that influence vulnerability semantics. By evaluating at three abstraction levels (single function, multiple functions, multiple files), we assess whether LLMs can leverage broader context, essential for deployment but unexplored in existing benchmarks.

LLM Benchmarks – From Scale to Quality and Leakage Control: Early benchmarks prioritized *scale*, collecting tens of thousands of samples but lacking vulnerability-specific metadata [28], [30]. The field evolved toward richer annotations [15]–[17] and CWE-wise analysis [18], [19], yet a critical gap remains: *training data contamination*. Most benchmarks rely on historical data that likely overlaps with model training corpora, artificially inflating metrics. Only limited efforts [19] have considered post-cutoff validation, yet none have used systematic temporal splits. Recent supervised detectors report substantially higher aggregate accuracy under different assumptions. For example, SecureFalcon fine-tunes a Falcon-derived classifier on FormAI/FalconVulnDB and reports 94% binary and 92% multiclass accuracy for C/C++ vulnerability classification [31], while Alam et al. [32] fine-tuned GPT-3.5 Turbo and GPT-4o Mini and reported 99% vulnerability-detection accuracy on Solidity. These results demonstrate

TABLE I
VD BENCHMARKS AT A GLANCE

Benchmark	Scale	CWE	Pairs (vuln vs patch)	Leakage	Granularity
[30] (2019)	27,318	–	✗	✗	Func
[15] (2022)	130	75	✓	✗	Snippet
[16] (2024)	1000	48	✓	✗	Program
[17] (2025)	294	77/86	✓	partial	Func.
[18] (2025)	5,000	25	partial	✗	Func.+project
[19] (2024)	228	8	✓	✓	Func.
LLMKernelBench	417	74	✓	✓	Func.+context

the value of task-specific fine-tuning and large task-specific datasets, but they are not directly comparable to our zero-shot evaluation of off-the-shelf LLMs on temporally separated Linux kernel CVEs.

LLMKernelBench addresses these limitations by prioritizing *quality over scale* and *leakage-aware* evaluation (Table I). We provide 314+103 curated CVE-linked samples across 74 CWE types with strict vulnerable-patched pairing. Critically, we introduce a temporal split: PBD (covering CVEs through 2024) versus LFD (covering 2025 post-training-cutoff CVEs), enabling a historical-versus-post-cutoff comparison rather than treating historical performance alone as evidence of generalization. Our hierarchy-aware evaluation and context-aware abstraction levels position LLMKernelBench as a *stress-test benchmark* for security analysis, complementing broader training corpora while addressing systematic limitations in rigor and contamination control.

III. BENCHMARKING APPROACH

LLMKernelBench is designed to assess pre-trained LLMs (see Table IV for the models evaluated) on SVD and CWE-label classification tasks, focusing on C code from the Linux kernel while being extensible to other programming languages. As illustrated in Figure 1, the framework is based on vulnerability metadata and code pairs extracted from Linux CVE commits, and includes prompts designed for SVD and CWE-label classification tasks. Raw model outputs are processed through the `LangChain VulnerabilityOutputParser`; for response-validity analysis, we additionally audit the raw text to distinguish genuine uncertainty from malformed or otherwise non-compliant output. The primary open-source SVD evaluation uses temperature 0 for deterministic decoding. The response-validity sweep averages the two complete temperature-0.1 runs, including GPT-4.1-mini, whereas the CWE-ranking analysis averages its three available ranking runs. We use binary classification metrics (precision, recall, accuracy, F1) for SVD and ranking metrics for CWE-label classification. This approach ensures benchmark reliability through representativeness (diverse real-world vulnerabilities), adaptability (an extendable methodology), portability (applicability across various LLMs), and fairness (standardized evaluation procedures). Section III-A details our dataset construction process. Section III-C defines metrics for both SVD and CWE-label classification tasks. Finally, Section III-D presents our standardized prompt templates.

A. Dataset

Our method systematically extracts pairs of vulnerable code blocks and their corresponding patched versions from

real-world Linux kernel commits. We focus on the Linux kernel due to its rich vulnerability history and critical security importance. We collect commit hashes, CVE identifiers, and CWE classifications from well-known vulnerability databases, including NVD [33], in line with established practices [34]. For each vulnerability-fixing commit, we retrieve the vulnerable code from its parent commit (pre-fix version) and the patched code from the commit itself (post-fix version).

Our extraction process identifies all security-related functions and non-functional elements (global variables, macros, type definitions, structs/unions, headers) that were directly modified in each commit. We include only functions with actual line-level changes, excluding unchanged code to avoid label noise. Non-functional elements are tracked separately as they often form critical vulnerability contexts that influence execution across multiple functions. This comprehensive approach assembles cohesive, vulnerable, and patched code blocks at multiple abstraction levels (single function, multiple functions, multiple files), enabling LLMs to analyze complete contextual information rather than isolated fragments.

After extraction, the dataset is divided into two. The PBD covers all vulnerabilities discovered on or before 2024, while the LFD covers the more recent 2025 vulnerabilities. This division enables us to evaluate the impact of potential data leakage on the model’s performance, thereby increasing our confidence in the generalizability of our findings.

Manual Review, Denoising, and Deduplication. Our automated extraction process allowed us to collect a total of 400 samples for PBD and 110 samples for LFD. After that, to ensure dataset integrity and label fidelity, both datasets were manually validated by two different reviewers. For this validation, a standardized protocol that defined inclusion criteria, exclusion criteria, and required evidence documentation was first defined. Then, the original datasets were split in half, and each half was assigned to a different reviewer. The reviewers followed the previously defined documentation to clean the dataset. Doubtful cases were marked for discussion and resolved through consensus sessions between the reviewers. The following validations were performed:

- **Denoising:** Removed samples and functions/files where the changes were not security-related.
- **Completeness check:** Ensured all relevant and security-impacting changes from each commit were included.
- **CWE frequency filtering:** CWEs that appeared only once were removed to improve representativeness.
- **Comment removal:** Stripped all code comments from both vulnerable and patched code blocks to eliminate potential biases and ensure that the LLMs relied solely on code structure and semantics for SVD.
- **Deduplication:** Removed duplicate or near-duplicate code samples (e.g., identical functions, commits, or cloned files) to prevent models from seeing the same or highly similar code during testing.
- **CWE labeling:** In the LFD, 40 samples lacked CWE identifiers in the NVD database; we manually labeled these samples, strictly adhering to NVD criteria for consistent labeling.

Table II summarizes the overall size and composition of the

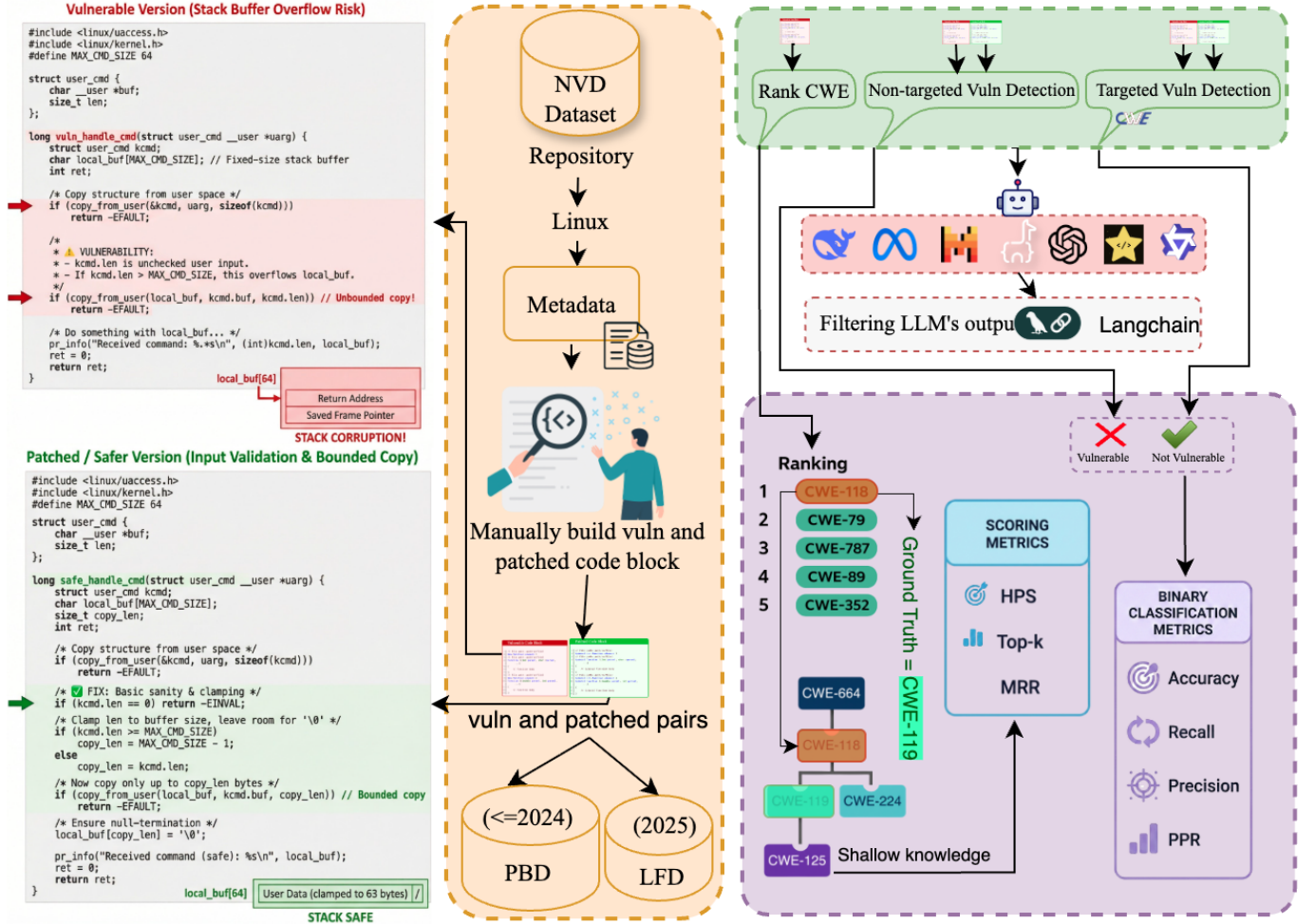


Fig. 1. LLMKernelBench Framework. Metadata (commit messages, diffs, function/file names) and vulnerable-patched code pairs are extracted from Linux CVE commits. Prompts are designed for SVD and CWE-label classification tasks in zero-shot settings. LLMs perform detection and classification, while LangChain parses raw model outputs and a response audit separates binary verdicts, genuine abstentions, and invalid outputs. Detection is evaluated using binary and ranking metrics.

PBD and LFD after this comprehensive cleaning process. The final PBD dataset comprises 314 samples spanning 74 distinct CWEs, while the final LFD dataset contains 103 samples belonging to 19 CWEs.

CWE Distribution and Label Space. We did not pre-select a preferred subset of CWEs. PBD retains the CWE labels associated with the sampled Linux-kernel CVEs, while 40 LFD samples lacking NVD CWE identifiers were manually labeled under NVD-aligned criteria. The resulting label space, spanning 74 distinct CWEs in PBD and 19 in LFD, therefore reflects the sampled kernel vulnerabilities rather than an artificially balanced category design. Our dataset spans diverse CWE categories organized according to the CWE-1000 hierarchical view [35]. Table III reports coverage against the ten fundamental CWE pillars: eight have nonzero coverage in the sampled data, while deprecated or otherwise unmapped labels are grouped as CWE-OTHER. CWE-664 accounts for 43.3% of PBD coverage, reflecting the prevalence of memory-related vulnerabilities in the Linux kernel. The distribution also includes 12.1% deprecated CWE identifiers (CWE-OTHER), representing historical vulnerability classifications that MITRE now marks as “PROHIBITED” for mapping.

TABLE II
LLMKERNELBENCH DATASET STATISTICS

Metric	PBD	LFD
Vulnerabilities	314	103
Unique CWEs	74	19
Files/Functions Changed (avg)	1.5 / 2.0	1.1 / 1.3
Vuln/Patched Lines (avg)	109.4 / 114.1	60.9 / 63.6
Lines Added/Deleted (avg)	17.6 / 9.0	7.0 / 3.5

TABLE III
CWE PILLAR DISTRIBUTION ACROSS PBD AND LFD.

CWE-ID	Pillar Name	Coverage (%)	
		PBD	LFD
CWE-664	Improper Control of Resource Lifetime	136 (43.3)	62 (60.2)
CWE-OTHER	Other/Unmapped CWEs	38 (12.1)	0 (0.0)
CWE-682	Incorrect Calculation	32 (10.2)	7 (6.8)
CWE-284	Improper Access Control	28 (8.9)	0 (0.0)
CWE-691	Insufficient Control Flow Management	27 (8.6)	11 (10.7)
CWE-707	Improper Neutralization	18 (5.7)	9 (8.7)
CWE-693	Protection Mechanism Failure	16 (5.1)	0 (0.0)
CWE-710	Improper Adherence to Coding Standards	11 (3.5)	14 (13.6)
CWE-703	Improper Check/Handling of Except. Conditions	9 (2.9)	0 (0.0)
CWE-435	Improper Interaction Between Components	0 (0.0)	0 (0.0)
CWE-697	Incorrect Comparison	0 (0.0)	0 (0.0)

B. Tasks

Based on this dataset, the models are evaluated across two different tasks: SVD and vulnerability type identification.

SVD. In this task, the models are given either a vulnerable code block (pre-fix versions from parent commits) or a non-vulnerable (patched) code block (post-fix versions from fixing commits). They are then asked to classify the code block as vulnerable (1), non-vulnerable (0), or to answer `not sure`. The uncertainty response is represented as `-1` in the structured evaluation data. This third option is used because: (1) real-world SVD often involves ambiguous cases where even security experts disagree; (2) it prevents LLMs from making forced binary decisions when evidence is insufficient, potentially reducing false positives and negatives; and (3) it mirrors practical security analysis workflows where analysts may need to flag code for further investigation rather than making definitive vulnerability determinations.

CWE-label classification. The CWE taxonomy organizes weaknesses into a hierarchical structure with four abstraction levels as shown in Figure 2: *Pillars* (highest level, 10 fundamental categories representing root security principles such as CWE-664 Improper Control of a Resource Through its Lifetime), *Classes* (general weakness types like CWE-119 Buffer Errors), *Bases* (specific weakness variants such as CWE-120 Buffer Copy without Checking Size of Input), and *Variants* (detailed platform-specific manifestations). This hierarchy is formalized in the CWE-1000 Research Concepts view [35].

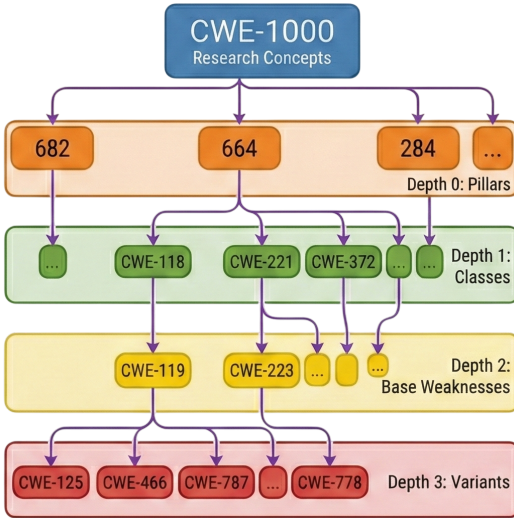


Fig. 2. CWE hierarchy.

For our CWE classification tasks, we employ a *hierarchical evaluation approach* that assesses model predictions across all taxonomy levels rather than focusing solely on exact matches. This design measures the taxonomic proximity of model outputs, distinguishing broad category-level matches from exact fine-grained identification without assuming that proximity alone demonstrates semantic understanding.

We introduce the HPS metric (§III-C), which assigns ordinal credit according to exact, immediate parent/child, sibling,

same-root, or unrelated relations in the CWE-1000 hierarchy. A prediction of CWE-121 (Stack-based Buffer Overflow) when the true answer is CWE-787 (Out-of-bounds Write) receives partial child credit, recording a related candidate despite missing the exact type. This approach addresses practical challenges in CWE classification: fine-grained labels (bases and variants) exhibit severe class imbalance, with many specific types having fewer than 10 examples in real-world datasets, rendering exact-match accuracy unreliable. Meanwhile, coarse-grained evaluation at only the pillar level (10 categories) would be too lenient to distinguish precise weakness identification from broad category prediction.

Our hierarchical scoring thus separates exact Top-1 identification from taxonomically related candidate output. High HPS with low Top-1 accuracy indicates related labels in the candidate set, not necessarily taxonomy-aware reasoning. This multi-level evaluation helps model developers locate error patterns and helps practitioners judge whether candidate labels are suitable only for routing or are accurate enough for diagnosis.

By considering both these tasks, our benchmark systematically evaluates LLMs in terms of their ability to (1) detect the presence of vulnerabilities through binary SVD classification, and (2) identify and prioritize the most likely underlying vulnerability types through CWE-label ranking.

C. Metrics

Due to the different nature of the two tasks, different metrics must be employed. These metrics, along with statistical significance testing, are discussed next.

1) *SVD*: To evaluate the models’ performance at SVD, we consider the following terms: True Positive (TP) refers to the number of vulnerable code samples that the model correctly identifies as vulnerable; False Positive (FP) is the number of non-vulnerable samples incorrectly predicted as vulnerable; True Negative (TN) represents the number of non-vulnerable samples correctly classified as non-vulnerable; while False Negative (FN) represents the number of vulnerable samples that the model fails to identify, incorrectly classifying them as non-vulnerable. These four quantities form the basis for calculating the performance metrics described below.

It is important to note that these metrics are computed only for cases in which the models provided a valid binary classification, i.e., vulnerable (1) or non-vulnerable (0). Genuine abstentions and invalid outputs are excluded. We distinguish these outcomes by auditing the raw response: a *genuine abstention* clearly selects `not sure` (either alone or as an unambiguous final verdict motivated by insufficient information), whereas an *invalid response* does not yield a single resolvable verdict, for example because it continues the input code, loops, or presents contradictory choices. Reporting these categories separately is critical because parser failure or prompt non-compliance should not be interpreted as model uncertainty. The following metrics are used:

- **Accuracy**: measures the proportion of correctly classified code samples among all samples:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (1)$$

As the dataset is relatively balanced between vulnerable and non-vulnerable samples, the accuracy accurately reflects the model’s overall performance without being dominated by one class. High accuracy is important as it indicates that the model performs well across both classes.

- **Recall** – measures the proportion of actual vulnerable code samples that were correctly identified by the model:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (2)$$

In this context, it represents the number of true vulnerabilities the LLM successfully detects or flags. High recall is crucial, as missing vulnerable samples could lead to the overlooking of potential security issues.

- **Precision** – measures the proportion of code samples predicted as vulnerable that are truly vulnerable:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3)$$

This metric indicates how many of the flagged samples are genuine vulnerabilities rather than false positives. High precision is important to reduce the number of incorrect alerts, which can waste time during manual inspection or downstream analysis.

- **Positive Prediction Rate (PPR)** – measures the proportion of all samples that the model classifies as vulnerable, regardless of correctness:

$$\text{PPR} = \frac{TP + FP}{TP + FP + TN + FN} \quad (4)$$

While this metric is not as critical as precision or recall, it provides valuable insight into the model’s potential bias toward one class. A higher PPR may indicate a tendency to over-predict vulnerabilities, whereas a lower PPR suggests under-prediction. Comparing the PPR across models helps identify whether a model systematically favors one class over the other, offering additional context for interpreting performance results.

Together, these metrics provide a comprehensive evaluation of classification performance, balancing correctness, sensitivity, and class bias.

2) *CWE-label classification*: For the task of CWE-label classification, the following metrics are employed:

- **Top-k Accuracy** [36]: Considers a prediction correct if the true vulnerability type appears among the model’s k highest-ranked predictions. We measure Top-1 to Top-5 accuracy to evaluate how well models detect and rank both CVEs and CWEs. This progressive evaluation is particularly valuable in security contexts where identifying the correct vulnerability category among top candidates enables more efficient remediation efforts, even when exact classification is challenging.
- **MRR** [37]: Evaluates ranking quality by calculating:

$$\frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i} \quad (5)$$

Where rank_i is the position of the correct answer in the ranked list for the i -th query. MRR rewards models that rank the correct vulnerability type higher, recognizing that, in security contexts, prioritizing findings is crucial.

- **HPS** scores a model’s outputs by how close they are to the ground-truth CWE in the hierarchy. We assign weights: exact = 1.0, parent = 0.8, child = 0.7, sibling = 0.6, same-root = 0.4, unrelated = 0.0. CWE-1000 is a directed acyclic graph rather than a strict tree because a weakness may have multiple parents or occur under multiple pillars. Parent and child therefore denote any immediate taxonomy edge, siblings share at least one immediate parent, and same-root labels share at least one top-level pillar. The HPS weighting scheme follows an ordinal decay aligned with the CWE hierarchy rather than modeling precise semantic distances. Higher weights reflect closer ancestry in the taxonomy, as confusing a vulnerability with an immediate parent or child category indicates greater taxonomic proximity than predicting an unrelated weakness. The specific values preserve hierarchical ordering and distinguish near-misses from unrelated predictions; they do not encode calibrated semantic margins. Formally, let Y_i be the set of ground-truth labels and $(\hat{y}_{i1}, \dots, \hat{y}_{ik_i})$ the valid predicted labels. Define $r(\hat{y}, y) \in \{\text{exact}, \text{parent}, \text{child}, \text{sibling}, \text{root}, \text{none}\}$ and a weight map $w(\cdot)$ as above. Each predicted label receives its best score against the ground-truth set,

$$s_{ij} = \max_{y \in Y_i} w(r(\hat{y}_{ij}, y)), \quad (6)$$

and the implemented dataset score averages across all valid predicted labels,

$$\text{HPS} = \frac{\sum_i \sum_{j=1}^{k_i} s_{ij}}{\sum_i k_i}. \quad (7)$$

Thus HPS measures taxonomic proximity of the candidate set, while Top- k accuracy and MRR separately measure exact inclusion and rank. For the supporting distance audit in Tables XV and XVIII, distance is the shortest path in the undirected CWE-1000 parent-child graph. We connect the ten top-level pillars through a virtual root so that cross-pillar errors have finite distances; unmapped or deprecated identifiers remain unmeasurable. The committed HPS and distance-analysis scripts preserve all parent and pillar memberships and regenerate the reported values directly from the three ranking runs. To ground this metric in a practical security context, consider a true CWE-787 (Out-of-bounds Write) with a predicted CWE-121 (Stack-based Buffer Overflow). CWE-121 is an immediate child of CWE-787 in the benchmark hierarchy and therefore receives the child weight of 0.7. By contrast, CWE-284 (Improper Access Control) is in a different root branch and receives 0.0. In SOC triage, the former can still route an alert toward buffer-handling review, but it must not be treated as an exact diagnosis.

By combining these metrics, we evaluate classification accuracy, ranking quality, and the taxonomic proximity of predicted vulnerability categories.

3) *Statistical Significance Testing*: To assess whether performance differences are statistically meaningful rather than random, we combine group-level tests, degradation metrics, and correlation analysis.

```

Check if the following C code block is
vulnerable.
Respond with ONLY one of these options:
- '1' if the code is vulnerable
- '0' if the code is not vulnerable
- 'not sure' if you are not sure if it is
vulnerable or not
Do not include any explanation or additional
text.
Code block:
{code}
Response:

```

Fig. 3. Verbatim vulnerability-detection template used in the production pipeline; patch verification substitutes the patched code block.

```

Instructions: Analyze this vulnerable code and return EXACTLY 5 CWE
identifiers as a JSON list. Return ONLY a JSON array in this exact format: ["
"CWE-XXX", "CWE-YYY", "CWE-ZZZ", "CWE-AAA", "CWE-BBB"]. NO
explanations, NO descriptions, NO additional text. Use only identifiers (CWE-
followed by numbers). Order by likelihood (most likely first).

Code Block Input:
{code}

```

Fig. 4. CWE ranking prompt.

Statistical test selection. To compare accuracy across abstraction levels (L1–L3), we first check normality (Shapiro-Wilk [38]) and homogeneity of variance (Levene). We then apply one-way ANOVA [39] when both hold, Welch’s ANOVA when variances differ, and the non-parametric Kruskal-Wallis H-test when normality fails. PBD satisfies both assumptions (Levene $p=0.812$), justifying ANOVA, whereas LFD Level 3 fails normality ($p=0.008$) and uses Kruskal-Wallis; $p<0.05$ denotes a significant effect of abstraction level on accuracy.

Degradation quantifies robustness to context as $\text{Accuracy}_{L1} - \text{Accuracy}_{L3}$: positive values denote decline with broader scope, negative values improvement, and near-zero values stable behavior across levels.

Pearson correlation ($r \in [-1, 1]$) measures the strength and direction of the linear relationship between two continuous variables (e.g., context window size vs. degradation), with significance assessed via its p-value.

Mann-Whitney U [40] is a non-parametric, rank-based test for two independent groups that is robust to outliers and skewed or unequal-sized samples. We use it to compare Simple (L1) versus Complex (L2+L3) accuracy in LFD, where the small L3 sample ($n=3$) and group imbalance (85 vs. 18) violate parametric assumptions. We pair it with the **rank-biserial correlation** [41] effect size,

$$r = 1 - \frac{2U}{n_1 \cdot n_2}, \quad (8)$$

where U is the Mann-Whitney statistic and n_1, n_2 the group sizes, to report practical magnitude alongside significance using Cohen’s thresholds [42] (negligible $|r| < 0.1$, small, medium, large $|r| \geq 0.5$). This matters under strong imbalance, where small differences can reach significance from sample size alone.

D. Prompt Templates

Our prompt templates for both SVD and CWE-label classification are designed with four key principles in mind: clear task specification, structured output formats, minimal

TABLE IV
LLM SPECS WITH CONTEXT WINDOW SIZES (TOKENS).

Model	Type	# Params	Context Window
CodeLlama	Code-spec.	7B	16K
StarCoder2	Code-spec.	7B	16K
Qwen3-Coder	Code-spec.	30B	262K
Llama3.1	General	8B	131K
Mistral (v0.3)	General	7B	32K
DeepSeek-R1	General	7B	131K
GPT-4.1-mini	General	—	1M

introduction of bias, and scalability across vulnerability types. We focus on benchmarking the inherent LLM capabilities rather than optimizing prompting strategies, and on evaluating baseline performance to inform future research directions. As shown in Figure 3, each prompt clearly states the expected task and output format, including explicit three-way response instructions (1, 0, or not sure; the uncertainty response is stored as -1 for evaluation), uses standardized templates for fair comparison across models, maintains essential code context while avoiding information leakage, and reflects real-world security analysis scenarios.

Representative prompt formats are shown in Figures 3 (SVD) and 4 (CWE-label classification).

IV. BENCHMARKING RESULTS

In this study, we leverage seven pre-trained LLMs to assess their capabilities in SVD and CWE-label classification tasks. Our selection comprises three code-specialized models (CodeLlama [43], StarCoder2 [44], and Qwen3-Coder [45]) explicitly trained on programming repositories, and four general-purpose models (Llama3.1 [9], Mistral [46], and DeepSeek-R1 [47], GPT-4.1-mini [48]) designed for broader language understanding tasks. This balanced selection enables us to investigate whether domain-specific training provides advantages over general language comprehension for security-critical tasks. We selected these models based on five key criteria: (1) their state-of-the-art performance on code understanding and generation benchmarks, (2) architectural diversity spanning both standard fully-dense models (e.g., CodeLlama) and Mixture of Experts (MoE) models (e.g., DeepSeek-R1) to ensure comprehensive evaluation, (3) varying parameter sizes (7B-30B and unknown for GPT-4.1-mini) and context window capacities (16K-1M tokens) to investigate how model scale and memory affect SVD capabilities, (4) balanced representation of code-specialized versus general-purpose training to assess domain expertise effects, and (5) inclusion of both smaller open-source and larger proprietary models, allowing for a fair comparison across accessibility and capability dimensions. The specifications of the models we used are detailed in Table IV.

A. RQ1: Overall SVD performance

For this research question, we first separate valid binary verdicts from genuine abstentions and invalid outputs. We then analyze classification performance in two settings: non-targeted, where no specific CWE is provided, and targeted, where the prompt includes the CWE label. Because the

validated response rates supplied for this audit are pooled over both settings and both datasets, Table V reports the overall temperature-0 response distribution rather than unsupported setting-specific estimates. Each available model contributes 1,668 responses (417 vulnerable/patched pairs evaluated in both settings).

TABLE V
RESPONSE VALIDITY AT TEMPERATURE 0, POOLED OVER
NON-TARGETED/TARGETED SETTINGS AND PBD/LFD. *Valid*: A
RESOLVABLE 0/1 VERDICT; *Genuine abstention*: A CLEAR UNCERTAINTY
VERDICT; *Invalid*: NON-COMPLIANT OR INDETERMINATE OUTPUT.
GPT-4.1-MINI WAS NOT RUN AT THIS TEMPERATURE.

Model	Valid (%)	Genuine abst. (%)	Invalid (%)
CodeLlama	98.20	0.00	1.80
DeepSeek-R1	100.00	0.00	0.00
GPT-4.1-mini	—	—	—
Llama3.1	100.00	0.00	0.00
Mistral	98.62	0.00	1.38
Qwen3-Coder-30B	99.46	0.06	0.48
StarCoder2	73.92	0.00	26.08

Genuine abstention is rare, while most non-answers are formatting failures. At temperature 0, Qwen3-Coder is the only model with observed genuine abstentions (0.06%). The other open-source models have no genuine abstentions. Invalid output is below 2% for CodeLlama, Mistral, and Qwen3-Coder, and absent for DeepSeek-R1 and Llama3.1. StarCoder2 is the clear exception: 26.08% of its outputs are invalid, primarily because the base completion model continues code or otherwise fails to emit one of the requested verdicts. The broader decoding sweep in Table XIX confirms that genuine abstention and invalid output are distinct behaviors: GPT-4.1-mini averages 2.40% genuine abstention across the two temperature-0.1 runs, while StarCoder2’s invalid rate reaches 26.08% at temperature 0.

1) Non-targeted Classification: LLMs performed no better than random classifiers. Table VI presents the performance metrics for the cases where the models provided a classification. All models exhibit accuracies close to 50%, irrespective of architecture, size, or context window size. This behaviour is observed on both PBD and LFD; the historical split does not show a clear aggregate advantage over the post-cutoff split. Because the splits differ in composition and size, this comparison constrains but does not isolate the causal effect of training-data exposure.

Consider CVE-2024-26599 (CWE-119, PBD), a clear out-of-bounds array access in a PWM driver function (Figure 5). The vulnerable code validates that `args->args_count` is either 1 or 2, but subsequently accesses `args->args[2]` when `args_count == 2`. Since arrays are zero-indexed, valid indices are only 0 and 1, making this an obvious out-of-bounds access. The patch simply replaces `args[2]` with `args[1]`. A human reviewer can immediately identify this as vulnerable because it violates a basic array bounds constraint. Yet the models still struggled to distinguish the vulnerable and patched versions reliably. In the first run, GPT-4.1-mini correctly classified both the vulnerable and patched samples. CodeLlama, Mistral, and DeepSeek classified both as vulnerable, while Llama3.1 classified both as safe. StarCoder2 produced an invalid response for the vulnerable version

```

1 // File: drivers/pwm/core.c
2 of_pwm_single_xlate(struct pwm_chip *chip,
3                     const struct of_phandle_args *args) {
4     struct pwm_device *pwm;
5
6     if (chip->of_pwm_n_cells < 1)
7         return ERR_PTR(-EINVAL);
8
9     // Validates that args_count is either 1 or 2
10    if (args->args_count != 1 && args->args_count != 2)
11        return ERR_PTR(-EINVAL);
12
13    pwm = pwm_request_from_chip(chip, 0, NULL);
14    if (IS_ERR(pwm))
15        return pwm;
16
17    pwm->args.period = args->args[0];
18    pwm->args.polarity = PWM_POLARITY_NORMAL;
19
20    // BUG: Accessing index 2 when count==2 (valid: 0,1)
21    - if (args->args_count == 2 && args->args[2] &
22      ↪ PWM_POLARITY_INVERTED)
23    + if (args->args_count == 2 && args->args[1] &
24      ↪ PWM_POLARITY_INVERTED)
25        pwm->args.polarity = PWM_POLARITY_INVERSED;
26    return pwm;
27 }

```

Fig. 5. CVE-2024-26599: out-of-bounds access; patch shown.

and classified the patched version as safe. This example is consistent with the aggregate class biases in Table VI: even when the local defect is explicit, several models preserve the same verdict across the vulnerable–patched pair rather than responding to the changed array index.

These results stand in sharp contrast to supervised detection tools evaluated under different assumptions: SecureFalcon [31] reports 94% binary accuracy on C/C++ code after training on FormAI/FalconVulnDB, and Alam et al. report 99% Solidity vulnerability-detection accuracy after fine-tuning GPT-3.5 Turbo and GPT-4o Mini [32]. These figures are not directly comparable because the studies use task-specific training datasets and different software domains, whereas our evaluation measures zero-shot, off-the-shelf performance and includes a post-cutoff split.

Models exhibit strong and consistent class biases rather than meaningful vulnerability detection behaviour. Examining Recall, Precision, and PPR reveals clear patterns across both datasets. In the non-targeted setting, CodeLlama, DeepSeek, Mistral, and Qwen3-Coder all achieve very high recall values (roughly 70–85%) together with high PPR values, indicating a strong tendency to classify most samples as vulnerable. However, their precision remains close to 50%, showing that these high recall scores are largely driven by overpredicting the positive class rather than correctly identifying vulnerabilities. In contrast, GPT-4.1-mini and Llama3.1 exhibit the opposite behaviour, with low recall and low PPR values across both datasets, indicating a strong bias toward predicting samples as non-vulnerable. Despite this conservative tendency, precision also remains close to random, meaning that even its positive predictions are unreliable. StarCoder2 appears comparatively more balanced, with recall and PPR values closer to 50%, but its overall accuracy still remains at chance level.

2) Targeted Classification: Providing the CWE label shifts model biases but does not improve classification per-

TABLE VI
PERFORMANCE METRICS FOR SVD IN NON-TARGETED AND TARGETED SETTINGS.

Model	Non-Targeted								Targeted							
	PBD (%)				LFD (%)				PBD (%)				LFD (%)			
	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR
CodeLlama	49.9	73.3	49.9	73.5	50.5	69.9	50.3	69.4	51.7	88.2	51.0	86.6	52.9	90.0	51.7	87.1
DeepSeek	49.6	81.3	50.1	80.1	50.0	77.7	50.0	77.7	49.6	87.3	50.2	86.1	51.9	86.4	51.3	84.5
GPT-4.1-mini	50.8	5.2	59.9	4.4	51.5	5.5	70.0	4.0	49.6	7.4	47.3	7.9	52.6	8.4	72.6	5.8
Llama3.1	50.1	31.6	50.1	31.5	51.0	29.4	51.6	28.5	49.4	27.3	48.9	27.9	47.7	15.5	43.6	17.8
Mistral	49.4	85.2	49.7	85.8	50.5	79.9	50.3	79.4	47.7	19.6	45.0	21.8	49.5	14.2	48.6	14.7
Qwen3-Coder	50.3	84.6	50.3	84.4	49.0	79.6	49.4	80.6	50.0	61.6	50.2	61.6	52.7	49.3	52.9	46.7
StarCoder2	49.0	52.2	47.0	52.7	49.9	48.8	49.5	49.3	49.2	61.4	48.4	62.2	53.6	67.0	50.8	62.9

TABLE VII
PER-CWE BREAKDOWN OF THE PERFORMANCE METRICS FOR SVD.

CWE Pillar	Non-Targeted								Targeted							
	PBD (%)				LFD (%)				PBD (%)				LFD (%)			
	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR	Acc.	Rec.	Prec.	PPR
CWE-284	48.8	59.4	48.7	60.6	—	—	—	—	47.0	46.4	47.0	49.4	—	—	—	—
CWE-664	50.7	60.0	50.5	59.1	49.7	54.0	49.9	54.3	49.7	50.4	49.8	50.6	50.8	46.6	50.6	45.8
CWE-682	49.0	59.6	49.2	60.6	51.4	63.0	51.3	61.6	51.1	49.0	51.3	47.9	49.4	46.5	48.9	47.2
CWE-691	48.7	55.2	48.8	56.5	49.3	57.8	49.3	58.5	50.3	49.4	50.2	49.1	51.1	48.5	51.2	47.4
CWE-693	52.7	60.5	51.8	57.7	—	—	—	—	48.9	48.6	48.8	49.7	—	—	—	—
CWE-703	50.7	57.0	50.2	56.2	—	—	—	—	48.6	51.3	48.6	52.8	—	—	—	—
CWE-707	50.1	60.5	50.6	60.6	54.9	60.5	53.9	55.5	50.2	57.6	50.0	57.3	53.1	53.9	52.7	50.7
CWE-710	48.6	57.4	48.8	58.5	50.7	56.4	50.3	55.7	47.7	34.4	47.4	36.4	54.9	41.9	55.6	37.1
CWE-OTHER	48.3	55.3	48.9	56.6	—	—	—	—	50.2	55.3	50.5	54.8	—	—	—	—

formance. In the targeted setting, Table VI (Targeted columns) summarizes the performance metrics. Accuracy remained near 50% for all models, so providing the CWE label did not produce a clear aggregate improvement. However, examining recall and PPR reveals notable bias shifts. CodeLlama and DeepSeek saw their positive class bias further amplified, with recall rising to around 88% and 87% on PBD respectively, while precision remained near 50%. Mistral exhibited the most dramatic reversal, flipping from a strong positive bias (recall: 85.2%) to a strong negative one (recall: 19.6%). Llama3.1’s existing negative bias deepened further, while Qwen3-Coder moved toward balance, with recall dropping from 84.6% to 61.6% on PBD. StarCoder2 and GPT-4.1-mini showed a modest increase in positive bias. Thus, CWE labels shift models in distinct and sometimes opposing directions without a consistent accuracy gain.

B. RQ2: Consistency across CWEs pillars

LLMs show weak, near-chance performance across CWE pillars, reinforcing that they are not yet capable of reliable SVD classification. In the non-targeted setting, performance varies slightly by pillar but remains close to random guessing. For PBD, accuracies range from 48.3% (CWE-OTHER) to 52.7% (CWE-693), with most pillars clustered near 50%. For LFD, results are only available for a subset of pillars (CWE-664/682/691/707/710) and accuracies range from 49.3% to 54.9%. Beyond accuracy, the other metrics show the same overall weakness: in PBD, recall spans 55.2%–60.5% and precision ranges from 48.7% to 51.8%.

The same trends can be seen for LFD and for the targeted setting, with models always remaining near chance across pillars, as summarized in Table VII.

C. RQ3: Consistency across granularity levels

Real-world vulnerabilities rarely exist in isolation. A single vulnerable function often depends on context from other functions, global variables, macros, or configuration settings spread across multiple files. To capture this reality, we evaluate models at three code abstraction levels: *Level 1* presents only the modified function in isolation; *Level 2* includes all functions modified in the security patch within a single file; and *Level 3* provides the complete context across multiple files.

Our comprehensive analysis across three abstraction levels reveals surprising patterns that challenge conventional assumptions about SVD complexity. We evaluated both PBD and LFD to ensure robust findings.

Abstraction Does Not Always Hinder Performance, But Individual Models Diverge. Mean accuracy remains similar across abstraction levels (PBD: 49.0–49.9%; LFD: 47.2–50.9%), and the omnibus tests do not reject equal group distributions. For PBD, one-way ANOVA yields $F=0.432$ ($p=0.6558$). For LFD, where normality assumptions fail (Shapiro-Wilk $p=0.008$ for L3), the Kruskal-Wallis test yields $H=2.995$ ($p=0.2237$). These non-significant results do not prove equivalence; they indicate that the study lacks evidence of a dataset-wide abstraction effect. Descriptively, LFD’s between-model standard deviation grows from 1.10% at Level 1 to 6.80% at Level 3. Table IX shows the corresponding heterogeneity: CodeLlama changes little in PBD but declines by 13.4% in LFD, DeepSeek declines by 3.2% in PBD and 4.7% in LFD, and StarCoder2 improves by 8.7% in LFD. Thus abstraction sensitivity is strongly model-specific in these samples.

Group Averages Differ Modestly, but Within-Group Variation Dominates. Code-specialized models average 0.4% degradation in PBD and 2.6% in LFD (1.5% across splits), compared with 1.1% and 4.5% for general-purpose models

TABLE VIII
OVERALL PERFORMANCE BY ABSTRACTION LEVEL: MEAN ACCURACY AND STANDARD DEVIATION ACROSS MODELS.

Abstraction Level	PBD		LFD	
	Mean (%)	Std. (%)	Mean (%)	Std. (%)
Level 1 (Single Function)	49.8	0.94	50.9	1.10
Level 2 (Multiple Functions)	49.9	2.77	50.6	3.65
Level 3 (Multiple Files)	49.0	1.98	47.2	6.80
Statistical tests: PBD: One-way ANOVA $F=0.432$, $p=0.6558$; LFD: Kruskal-Wallis $H=2.995$, $p=0.2237$				

TABLE IX
MODEL ROBUSTNESS BY ABSTRACTION SENSITIVITY. DEGRADATION = L1 - L3 ACCURACY; NEGATIVE INDICATES IMPROVEMENT. RANKED BY AVERAGE DEGRADATION (WORST → BEST).

Rank (worst → best ↓)	Model	Type	Context Window	Degradation (%)		
				PBD	LFD	Avg
1	Codellama	Code-Spec.	16K	0.2	13.4	6.8
2	Llama	General	128K	-2.1	11.6	4.8
3	Deepseek	General	64K	3.2	4.7	4.0
4	Qwen3	Code-Spec.	32K	0.1	3.2	1.6
5	Mistral	General	32K	3.2	-0.3	1.4
6	GPT-4.1-mini	General	1M	0.2	1.9	1.0
7	StarCoder	Code-Spec.	16K	1.0	-8.7	-3.8
Code-Specialized Avg.				0.4	2.6	1.5
General-Purpose Avg.				1.1	4.5	2.8

(2.8% across splits). With only three and four models per group, these descriptive averages do not establish a specialization effect. Within-group variation is larger: StarCoder2 improves by 3.8% on average as abstraction increases, while CodeLlama degrades by 6.8%; GPT-4.1-mini averages 1.0% degradation, whereas Llama3.1 averages 4.8%. The evidence therefore supports model-specific sensitivity more strongly than a general code-specialization advantage.

Context Window Size Does Not Guarantee Robustness. GPT-4.1-mini, despite having the largest context window (1M tokens), shows more degradation than StarCoder2 (1.0% vs. -3.8%), which has a 16K-token window. Pearson correlation is not statistically significant in either split (PBD: $r=-0.365$, $p=0.635$; LFD: $r=-0.592$, $p=0.408$). With seven models, this analysis only shows that context-window size alone does not explain the observed robustness pattern.

Task-Specific Patterns Are Descriptive, Not Statistically Conclusive. Mann-Whitney U tests in Table X compare Simple (L1) with Complex (L2 and L3) LFD samples. Non-targeted tasks show a medium estimated effect ($U=69$, $p=0.1442$, $r=0.326$), while targeted tasks show a small effect ($U=56$, $p=0.6272$, $r=0.114$); neither comparison is statistically significant. Descriptively, targeted tasks exhibit 0.4% (PBD) and 2.2% (LFD) L1-to-L3 degradation, compared with 1.2% and 5.2% for non-targeted tasks. This pattern motivates further study of whether CWE context helps focus analysis in larger code scopes, but the present data do not establish that mechanism.

The imbalanced distribution of abstraction levels in LFD (L1: 82.5%, L2: 14.6%, L3: 2.9%) reflects natural real-world patterns but limits statistical power for fine-grained analysis. Our comparison in Table X addresses this limitation by using appropriate nonparametric methods and reporting effect sizes,

TABLE X
TASK-SPECIFIC ACCURACY ACROSS ABSTRACTION LEVELS AND L1 → L3 CHANGE. EXCLUDED ABSTENTION CASES.

Task	PBD Accuracy (%)				LFD Accuracy (%)			
	L1	L2	L3	Δ (L1→L3)	L1	L2	L3	Δ (L1→L3)
Non-Targeted	50.0	50.4	48.8	1.2	50.4	49.9	45.2	5.2
Targeted	49.6	49.4	49.2	0.4	51.4	51.4	49.2	2.2
Statistical tests (LFD, Simple vs Complex): Non-Targeted: Mann-Whitney $U=69$, $p=0.1442$, $r=0.326$ (medium effect) Targeted: Mann-Whitney $U=56$, $p=0.6272$, $r=0.114$ (small effect)								

but future work should include the targeted collection of multi-component, multi-file vulnerabilities to enable more robust three-way comparisons. Nevertheless, our finding that the abstraction level does *not* consistently affect average performance (as shown in Tables IX and X), combined with the dramatic model-specific divergence in robustness, provides valuable insights for deployment strategies for vulnerability detection.

D. RQ4: CWE Ranking Performance

Identifying the specific vulnerability type is critical for security practitioners, as knowing the exact CWE category helps developers understand the root cause and apply appropriate fixes. This task is challenging because the CWE system is hierarchical, with over 900 weakness types organized into four levels: Pillars, Classes, Base weaknesses, and Variants. We retain the sampled fine-grained CWE identifiers for ranking and use root pillars only to organize coverage and hierarchy-aware relations.

We evaluate models on the CWE ranking task, where models rank their top-5 most likely CWE predictions for a given vulnerable code snippet. Performance is measured using Top-k accuracy, MRR, and HPS, which rewards taxonomically related predictions based on the CWE hierarchy. Table XI summarizes results across both datasets. We opted for a ranking formulation instead of binary per-CWE classification because the dataset spans 74 distinct CWE identifiers, many with fewer than 10 examples, making individual binary classifiers statistically unreliable. The Top- k accuracy and HPS metrics together capture exact identification and partial taxonomic credit.

HPS Reveals Limited Taxonomic Proximity Despite Low Exact Accuracy. Table XI reveals that precise CWE identification is far harder than binary vulnerability detection. Even the best model, GPT-4.1-mini, achieves only 3.3% Top-1 accuracy in PBD and 15.6% in LFD. DeepSeek manages less than 1% Top-1 accuracy in both datasets (0.3% PBD, 0.0% LFD), while CodeLlama achieves 0.5% in PBD and 1.6% in LFD. Top-5 performance is higher: GPT-4.1-mini reaches 52.7% in LFD, showing that the exact label is sometimes present among its candidates even when not ranked first. CWE ranking exhibits mixed patterns: GPT-4.1-mini and Qwen3-Coder improve in LFD (approximately 5× and 3× better Top-1 accuracy, respectively), while StarCoder2 and DeepSeek reach 0.0% Top-1 in LFD. The HPS-Top-1 gap (19.3% vs. 3.3% for GPT-4.1-mini in PBD; Table XI) records some taxonomically related candidates, but only 13.9% (289/2,077) of incorrect Top-1 predictions are within hierarchy distance

TABLE XI
CWE RANKING ACROSS PBD AND LFD, AVERAGED ACROSS THREE RUNS. COLUMNS REPORT TOP- k ACCURACY, MRR, AND HPS.

Model	PBD: CWE Top-k (%)					LFD: CWE Top-k (%)					MRR (%)		HPS (%)	
	T_1	T_2	T_3	T_4	T_5	T_1	T_2	T_3	T_4	T_5	PBD	LFD	PBD	LFD
CodeLlama	0.5	0.8	1.0	1.0	1.3	1.6	2.3	2.3	2.6	6.5	0.9	3.0	6.5	8.9
DeepSeek	0.3	0.5	0.7	0.8	1.2	0.0	0.3	0.3	0.3	0.3	0.6	0.2	6.3	10.5
GPT-4.1-mini	3.3	5.8	7.6	9.5	10.7	15.6	26.2	41.5	51.7	52.7	5.8	28.8	19.3	31.1
Llama3.1	1.2	2.1	4.4	6.5	7.3	1.3	5.5	9.1	11.7	14.9	3.1	5.9	10.7	15.9
Mistral	1.1	2.4	3.2	5.3	6.9	5.2	7.2	8.8	9.8	10.4	2.9	7.1	9.1	10.7
Qwen3-Coder-30B	2.2	4.8	6.7	7.8	10.0	6.2	14.6	18.2	23.1	31.2	4.9	14.4	15.4	21.9
StarCoder2	1.3	3.1	4.4	6.5	6.7	0.0	0.6	0.6	3.6	5.9	3.2	1.6	19.0	26.9
Average	1.4	2.8	4.0	5.3	6.3	4.3	8.1	11.5	14.7	17.4	3.0	8.7	12.3	18.0

TABLE XII

MICRO/MACRO TOP-5 EXACT-INCLUSION ACCURACY ACROSS CWE TYPES. GAPS ARE COMPUTED FROM UNROUNDED THREE-RUN MEANS AND MAY DIFFER BY 0.1 POINT FROM SUBTRACTING THE DISPLAYED VALUES.

Model	PBD			LFD		
	Micro (%)	Macro (%)	Gap (pp)	Micro (%)	Macro (%)	Gap (pp)
CodeLlama	2.1	2.2	-0.1	7.5	3.3	4.2
DeepSeek	1.2	1.0	0.1	0.7	0.5	0.1
Llama3.1	7.3	5.9	1.4	14.9	14.7	0.2
Mistral	6.9	5.5	1.5	10.4	11.5	-1.1
Qwen3-Coder	10.0	7.7	2.4	31.2	19.7	11.5
StarCoder2	6.8	5.5	1.3	6.5	2.6	4.0
GPT-4.1-mini	10.7	10.7	0.0	52.7	25.3	27.5
Average	6.4	5.5	0.9	17.7	11.1	6.6

3 (Table XVIII). Such near misses may help route manual review, but the low rate does not establish broad semantic understanding. However, the large HPS-to-Top-1 ratios ($5.9\times$ to $17.2\times$ in PBD; Table XVII) and systematic CWE-119 bias (44.2% appearance rate, with Llama reaching 98.6%, Table XVI) suggest this may reflect taxonomy pattern-matching rather than genuine semantic understanding.

Frequent Vulnerability Types Are Associated with Higher Aggregate Performance. Table XII shows that aggregate scores are generally higher when frequent CWE types receive more weight. As defined in §III-C, micro accuracy weights all samples equally (thereby emphasizing frequent types), while macro accuracy averages performance across CWE classes equally. On average, the micro-macro gap is 0.9 points in PBD and 6.6 points in LFD. This pattern is consistent with sensitivity to class frequency, but the difference between splits may also reflect their different class composition and the smaller LFD sample. Individual models show stronger effects: GPT-4.1-mini achieves 52.7% micro but 25.3% macro in LFD (27.4-point gap), while Qwen3-Coder shows an 11.5-point gap (31.2% vs. 19.7%). Mistral instead has a small negative LFD gap (-1.1), indicating slightly higher average performance across classes than across samples. These descriptive differences do not identify a particular internal model mechanism. Per-CWE estimates, especially for classes with fewer than approximately 20 samples, should therefore be treated as indicative rather than statistically definitive.

Performance Concentrates on Input-Validation and Memory-Related CWEs. Table XIII shows the highest PBD averages for CWE-119 (Buffer Errors, 44.8%), CWE-20 (Improper Input Validation, 34.2%), CWE-122 (Heap Overflow, 34.0%), CWE-129 (Improper Array Index Validation, 21.9%),

TABLE XIII

TOP-5 AND BOTTOM-5 CWE DETECTION PERFORMANCE (AVERAGED ACROSS ALL MODELS AND THREE RUNS).

PBD Benchmark		LFD Benchmark	
CWE ID	Acc. (%)	CWE ID	Acc. (%)
<i>Top Performers</i>		<i>Top Performers</i>	
CWE-119 ($\uparrow_1$)	44.8	CWE-20 ($\uparrow_1$)	34.5
CWE-20 ($\uparrow_2$)	34.2	CWE-787 ($\uparrow_2$)	33.3
CWE-122 ($\uparrow_3$)	34.0	CWE-129 ($\uparrow_3$)	29.5
CWE-129 ($\uparrow_4$)	21.9	CWE-125 ($\uparrow_4$)	25.5
CWE-190 ($\uparrow_5$)	21.4	CWE-190 ($\uparrow_5$)	20.0
<i>Bottom Performers</i>		<i>Bottom Performers</i>	
CWE-191 ($\downarrow$)	0.0	CWE-476 ($\downarrow$)	14.5
CWE-276 ($\downarrow$)	0.0	CWE-835 ($\downarrow$)	3.2
CWE-665 ($\downarrow$)	0.0	CWE-667 ($\downarrow$)	2.4
CWE-755 ($\downarrow$)	0.0	CWE-908 ($\downarrow$)	0.0
CWE-763 ($\downarrow$)	0.0	CWE-401 ($\downarrow$)	0.0

and CWE-190 (Integer Overflow, 21.4%). Five reported CWEs remain undetected at 0%: CWE-191 (Integer Underflow), CWE-276 (Incorrect Default Permissions), CWE-665 (Improper Initialization), CWE-755 (Improper Handling of Exceptional Conditions), and CWE-763 (Release of Invalid Pointer). LFD shows a related concentration, with CWE-20 reaching 34.5%. These descriptive results are compatible with several explanations, including class frequency, training exposure, and the visibility of local code patterns; the benchmark does not disentangle those causes, especially for rare CWEs.

CWE-119 Acts as a Frequent Destination for Diverse Errors. Table XIV shows that all top-eight PBD confusion pairs target CWE-119 (Buffer Errors), spanning heap overflow (CWE-122, 17.7 instances), incorrect permissions (CWE-276, 15.3), buffer copy (CWE-120, 13.7), use-after-free (CWE-416, 13.7), and several resource-management labels. LFD shows a different pattern: CWE-416 (Use After Free) is the most frequent source label in six of the eight leading confusion pairs and maps to several memory-related predictions. These counts document concentration in the error distribution without establishing why the models choose those labels.

HPS Validation. The HPS metric assumes that taxonomy relations provide a useful proxy for semantic similarity between vulnerability types. To examine that proxy, we analyzed 2,107 Top-1 CWE predictions across all models (Table XV). Only 13.9% (289/2,077) of incorrect predictions were hierarchically close (distance ≤ 3), so most wrong predictions are not near the true label in the hierarchy (Table XVIII). For example, a true CWE-119 with a predicted CWE-121 has hierarchy distance 2, recording a buffer-family near miss without proving

TABLE XIV
TOP-8 MISCLASSIFICATION PATTERNS SHOWING CWE-119 AS A
SEMANTIC MAGNET IN PBD AND CWE-416 CONFUSION DOMINANCE IN
LFD. COUNTS ARE AVERAGED ACROSS THREE RUNS.

PBD			LFD		
True	Predicted	Count	True	Predicted	Count
CWE-122	CWE-119	17.7	CWE-416	CWE-119	59.7
CWE-276	CWE-119	15.3	CWE-125	CWE-119	31.7
CWE-120	CWE-119	13.7	CWE-476	CWE-119	23.0
CWE-416	CWE-119	13.7	CWE-416	CWE-121	20.0
CWE-772	CWE-119	13.7	CWE-416	CWE-123	19.7
CWE-284	CWE-119	12.7	CWE-416	CWE-20	17.7
CWE-399	CWE-119	12.0	CWE-416	CWE-125	12.7
CWE-665	CWE-119	12.0	CWE-416	CWE-1234	12.3

semantic reasoning. However, we observed systematic bias toward predicting CWE-119: this general category appears in 44.2% of all top-5 predictions, with Llama reaching 98.6% (Table XVI). Frequent inclusion of this broad category can raise HPS without improving exact identification. Additionally, the CWE hierarchy does not always reflect semantic similarity: CWE-129 (Array Index) and CWE-119 (Buffer Errors) are both bounds-related but have distance 7, so a plausible confusion can receive low HPS credit. The large HPS-to-Top-1 ratios (Table XVII) can arise from repeated broad or related candidates and must not be interpreted as proof of semantic understanding. We therefore interpret HPS narrowly as taxonomic proximity and recommend validation through human similarity ratings correlated with hierarchy distance.

V. DISCUSSION

Our conclusions apply specifically to off-the-shelf, pre-trained LLMs in a zero-shot setting with standardized prompts; we do not assess task-specific fine-tuning, few-shot prompting, chain-of-thought reasoning, external tools, or hybrid pipelines. They should thus be read not as fundamental limitations of LLMs as a class, but as evidence that current baseline usage is insufficient for reliable vulnerability detection in high-assurance settings. Model behavior is described observationally: references to tendencies, biases, or memorization characterize consistent prediction patterns, not internal intent or mechanisms.

Key Findings. Our benchmark exposes substantial limits in zero-shot LLMs vulnerability analysis. Low CWE accuracy (best Top-1 only 3.3% on PBD and 15.6% on LFD), strong class biases, and frequent identical verdicts for vulnerable-patched pairs are inconsistent with reliable code-sensitive classification. Providing CWE context does not consistently help and sometimes hurts. These observations support using the evaluated models as assistive tools under expert supervision rather than autonomous analyzers; they do not by themselves identify the models’ internal reasoning mechanism.

Deployment cost at multi-file context. Median Level 3 prompts are $\approx 2.7\times$ longer than Level 1 across all seven models. In the two released GPT-4.1-mini temperature-0.1 logs, the median sequential completion interval increases only slightly from 0.51 s at Level 1 to 0.53 s at Level 3 for the non-targeted PBD task; these timestamps include API and

TABLE XV
HIERARCHY DISTANCE ANALYSIS FOR INCORRECT CWE PREDICTIONS
ACROSS MODELS, AVERAGED ACROSS THREE RUNS. “CLOSE ≤ 3 ”
INDICATES PREDICTIONS WITHIN HIERARCHY DISTANCE 3 OF THE TRUE
CWE. STATISTICS COMPUTED OVER N PREDICTIONS PER MODEL.
PER-MODEL COUNTS ARE INDEPENDENTLY ROUNDED FROM THREE-RUN
AVERAGES, SO DISPLAYED ROWS NEED NOT SUM EXACTLY TO THE
OVERALL ROW.

Model	N	Wrong	Close ≤ 3	Close (%)	Avg Dist
GPT-4.1-mini	306	296	63	21.4	4.57
Starcode	308	304	59	19.3	4.47
Llama	312	308	56	18.3	4.53
Mistral	310	306	31	10.1	4.78
Codellama	311	309	30	9.8	5.83
Qwen3-Coder	310	303	27	9.0	5.89
Deepseek	250	249	22	8.7	4.96
Overall	2107	2077	289	13.9%	5.00

TABLE XVI
FREQUENCY OF CWE-119 APPEARING IN TOP-5 PREDICTIONS ACROSS
MODELS, INDICATING SYSTEMATIC BIAS TOWARD THIS GENERAL BUFFER
ERRORS CATEGORY. COUNTS AVERAGED ACROSS THREE RUNS.

Model	CWE-119 in Top-5	Total Pred.	Rate (%)
Llama	308	312	98.6
Starcode	249	308	80.8
GPT-4.1-mini	215	306	70.3
Qwen3-Coder	99	310	31.8
Deepseek	35	250	14.1
Mistral	21	310	6.7
Codellama	6	311	1.8
Overall	932	2107	44.2

local orchestration overhead and are not a controlled service-level benchmark. For a self-hosted Llama3.1-8B on L40S, the artifact-based projection rises from 2.5 s to 6.8 s. At the pricing snapshot used in our analysis^{1,2}, a typical eight-output-token GPT-4.1-mini request costs approximately \$0.53 per 1,000 Level 3 predictions; \$1.34 is a conservative upper bound if every response reaches the 512-token cap. The corresponding Llama3.1-8B estimate is \$1.61, so CI/CD operators should reserve multi-file context for diff-triggered review batches rather than indiscriminate continuous scanning.

Future Directions. Future work should validate HPS through human expert studies (e.g., having practitioners rate CWE-pair similarity on a 1–5 scale and correlating it with hierarchy distance via Spearman’s ρ). LLMKernelBench further enables: (1) analyzing LLMs’ reasoning to locate security-comprehension weaknesses [49], [50]; (2) expanding to other languages (Python, Java, Rust, Go); (3) hybrid LLM-agent architectures integrating static/dynamic analysis; and (4) examining whether LLMs introduce vulnerabilities when generating patches.

VI. THREATS TO VALIDITY

Although the methodology and experiments in this study were carefully designed and executed, some potential threats

¹GPT-4.1-mini pricing at launch: <https://openai.com/index/gpt-4-1/>, retrieved 2026-05-06.

²RunPod community-cloud GPU pricing: <https://www.runpod.io/pricing>, retrieved 2026-05-06.

TABLE XVII

GAP BETWEEN HPS AND TOP-1 ACCURACY SHOWING POTENTIAL TAXONOMY PATTERN-MATCHING. RATIO INDICATES HOW MANY TIMES LARGER HPS IS COMPARED TO TOP-1 AND IS COMPUTED FROM UNROUNDED THREE-RUN MEANS. DATA FROM TABLE XI.

Model	PBD			LFD		
	Top-1 (%)	HPS (%)	Ratio (×)	Top-1 (%)	HPS (%)	Ratio (×)
StarCoder	1.3	19.0	14.6	0.0	26.9	∞
Deepseek	0.3	6.3	17.2	0.0	10.5	∞
Codellama	0.5	6.5	12.2	1.6	8.9	5.4
Llama	1.2	10.7	9.1	1.3	15.9	12.2
Mistral	1.1	9.1	8.5	5.2	10.7	2.0
Qwen3-Coder	2.2	15.4	6.8	6.2	21.9	3.5
GPT-4.1-mini	3.3	19.3	5.9	15.6	31.1	2.0

TABLE XVIII

DISTRIBUTION OF HIERARCHY DISTANCES FOR INCORRECT CWE PREDICTIONS WITH MEASURABLE DISTANCES ($N \approx 1708$ OF ≈ 2077 INCORRECT PREDICTIONS PER RUN ON AVERAGE; COUNTS AVERAGED ACROSS THREE RUNS). 16.9% OF THESE ERRORS FALL WITHIN PROXIMITY ≤ 3 (13.9% OF ALL INCORRECT PREDICTIONS).

Distance	Count	Prop. (%)	Cumul. (%)
1	46.0	2.7	2.7
2	123.3	7.2	9.9
3	119.3	7.0	16.9
4	316.7	18.5	35.4
5	421.7	24.7	60.1
6	365.3	21.4	81.5
7	219.0	12.8	94.4
8	72.7	4.3	98.6
9	23.7	1.4	100.0
Total ≤ 3	288.7	16.9	—

have to be discussed.

Internal Validity. The non-deterministic nature of LLMs is a potential threat; the primary open-source evaluation uses temperature 0 for deterministic decoding (§III). To assess sensitivity to stochastic decoding, we also analyze two complete temperature-0.1 runs and a temperature-1.0 run; GPT-4.1-mini is available only in the temperature-0.1 runs. Table XIX shows that response validity can change substantially with decoding, especially for StarCoder2. Prompt formulation can also affect outcomes, so we use standardized zero-shot prompts rather than optimized prompt engineering, establishing a consistent and fair baseline. Some models may have encountered PBD data during pretraining; to estimate this effect, we introduced the LFD dataset of vulnerabilities disclosed after the models’ training cutoffs and compare behavior across both splits.

Manual annotation introduces subjectivity, which we mitigated with clear guidelines, strict reviewer adherence, and consensus discussion of uncertain cases. Extracting only modified functions may omit surrounding context; we address this through three abstraction levels, validated by the distinct performance patterns in our RQ3 analysis (§IV-C). Finally, all tasks focus on static code reasoning and do not capture dynamic execution or runtime verification, which isolates reasoning capability but may limit interpretation in settings requiring dynamic analysis.

External Validity. Our analysis is restricted to Linux kernel C code, which may limit generalizability to other languages or domains; nonetheless, the kernel’s scale, complexity, and security-critical role make it a meaningful stress-test domain. Models released after our evaluation are not included, and the observed consistency across the seven evaluated systems does

not guarantee transfer to newer architectures or other software domains.

Construct Validity. HPS relies on a manually defined weighting scheme to quantify hierarchical proximity within the CWE taxonomy. The weights encode an ordinal taxonomic ordering rather than calibrated semantic distances, so absolute HPS magnitudes should be interpreted cautiously. We do not perform a sensitivity analysis over alternative weights; model rankings and gaps could change under another defensible scheme.

CWE assignment, particularly at fine-grained base and variant levels, involves inherent subjectivity even among experts. This is especially relevant for the LFD, where some CWEs were manually assigned following NVD-aligned guidelines, and formal inter-annotator agreement was not computed. Consequently, part of the low Top-1 accuracy in fine-grained CWE classification may reflect label ambiguity rather than purely model failure. To mitigate this, we emphasize hierarchy-aware evaluation and relative performance trends, which are more robust to label noise.

Conclusion Validity. Our conclusions are based on pre-trained LLMs evaluated in a zero-shot setting with standardized prompts. While this enables fair and reproducible comparison, it does not capture the effects of prompt adaptation, few-shot examples, or task-specific fine-tuning. Thus, the observed limitations reflect the reliability of baseline LLM deployment rather than definitive bounds on LLM security reasoning.

Although appropriate statistical tests and effect sizes are reported, the LFD’s limited size constrains statistical power, particularly at higher abstraction levels. As a result, some differences may be sensitive to sampling variance, and non-significant results should not be interpreted as evidence of equivalence. The LFD is intended to reveal robustness trends under leakage-free conditions rather than support fine-grained statistical generalization. Likewise, per-CWE estimates for classes with fewer than approximately 20 samples are exploratory and should not support statistically definitive class-level claims. Finally, descriptions of model behavior (e.g., bias or memorization) refer to observed prediction patterns and do not imply internal model intent or mechanisms.

VII. CONCLUSION

This paper presents LLMKernelBench, a benchmark for evaluating LLM capabilities in SVD and CWE classification using 417 real-world Linux kernel vulnerabilities. Our evaluation of seven models reveals substantial limitations: binary detection achieves near-random performance ($\sim 50\%$ accuracy), CWE classification remains weak (best: 15.6% Top-1, 52.7% Top-5), and models exhibit strong prediction biases. Abstraction analysis shows model-specific responses, with some improving on added context while others degrade by up to 13.4%. On LFD, abstraction robustness is model-specific rather than tied to specialization (code-specialized 2.6% vs. general-purpose 4.5% average degradation), while the larger micro-to-macro CWE-accuracy gap (0.9 points in PBD and 6.6 in LFD) is consistent with class-frequency sensitivity but may also reflect split composition. The evaluated zero-shot models

therefore require expert supervision, and LLMKernelBench provides a reproducible foundation for building more reliable detectors.

REFERENCES

- [1] CVE Details, “Vulnerability details and information,” <https://www.cvedetails.com/>, 2024, accessed: 2024-06-19.
- [2] K. Huang, Z. Xu, S. Yang, H. Sun, X. Li, Z. Yan, and Y. Zhang, “A survey on automated program repair techniques,” *arXiv preprint arXiv:2303.18184*, 2023.
- [3] Q. Zhang, C. Fang, Y. Ma, W. Sun, and Z. Chen, “A survey of learning-based automated program repair,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 2, pp. 1–69, 2023.
- [4] V. J. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, “The art, science, and engineering of fuzzing: A survey,” *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2019.
- [5] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, “Evaluating fuzz testing,” in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 2123–2138.
- [6] J. D. Pereira, J. R. Campos, and M. Vieira, “Machine learning to combine static analysis alerts with software metrics to detect security vulnerabilities: An empirical study,” in *2021 17th European Dependable Computing Conference (EDCC)*. IEEE, 2021, pp. 1–8.
- [7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [8] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [9] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [10] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, “Codebert: A pre-trained model for programming and natural languages,” *arXiv preprint arXiv:2002.08155*, 2020.
- [11] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, “Competition-level code generation with alphacode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [12] N. Alshahwan, J. Chheda, A. Finegenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, “Automated unit test improvement using large language models at meta,” *arXiv preprint arXiv:2402.09171*, 2024.
- [13] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software testing with large language models: Survey, landscape, and vision,” *IEEE Transactions on Software Engineering*, 2024.
- [14] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, “Large language models for software engineering: Survey and open problems,” in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. IEEE, 2023, pp. 31–53.
- [15] M. L. Siddiq and J. C. Santos, “Securityeval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques,” in *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 2022, pp. 29–33.
- [16] Y. Liu, L. Gao, M. Yang, Y. Xie, P. Chen, X. Zhang, and W. Chen, “Vuldetbench: Evaluating the deep capability of vulnerability detection with large language models,” *arXiv preprint arXiv:2406.07595*, 2024.
- [17] Y. Sun, D. Wu, Y. Xue, H. Liu, W. Ma, L. Zhang, M. Shi, and Y. Liu, “Llm4vuln: A unified evaluation framework for decoupling and enhancing llms’ vulnerability reasoning,” *arXiv preprint arXiv:2401.16185*, 2024.
- [18] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, “Understanding the effectiveness of large language models in detecting security vulnerabilities,” in *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2025, pp. 103–114.
- [19] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, “Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks,” in *IEEE Symposium on Security and Privacy*, 2024.
- [20] Z. Gao, H. Wang, Y. Zhou, W. Zhu, and C. Zhang, “How far have we gone in vulnerability detection using large language models,” *arXiv preprint arXiv:2311.12420*, 2023.
- [21] E. Iannone, R. Guadagni, F. Ferrucci, A. De Lucia, and F. Palomba, “The secret life of software vulnerabilities: A large-scale empirical study,” *IEEE Transactions on Software Engineering*, vol. 49, no. 1, pp. 44–63, 2022.
- [22] R. Croft, M. A. Babar, and M. M. Kholoosi, “Data quality for software vulnerability datasets,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 121–133.
- [23] MITRE Corporation, “Common Weakness Enumeration (CWE),” <https://cwe.mitre.org/>, 2024, accessed: 2025-10-18.
- [24] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep learning based vulnerability detection: Are we there yet?” *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.
- [25] X. Wang, S. Wang, P. Feng, K. Sun, and S. Jajodia, “Patchdb: A large-scale security patch dataset,” in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2021, pp. 149–160.
- [26] Y. Zheng, S. Pujar, B. Lewis, L. Buratti, E. Epstein, B. Yang, J. Laredo, A. Morari, and Z. Su, “D2a: A dataset built for ai-based vulnerability detection methods using differential analysis,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2021, pp. 111–120.
- [27] C. Ni, L. Shen, X. Yang, Y. Zhu, and S. Wang, “Megavul: Ac/c++ vulnerability dataset with comprehensive code representations,” in *Proceedings of the 21st International Conference on Mining Software Repositories*, 2024, pp. 738–742.
- [28] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” *Advances in neural information processing systems*, vol. 32, 2019.
- [29] G. Bhandari, A. Naseer, and L. Moonen, “Cvefixes: automated collection of vulnerabilities and their fixes from open-source software,” in *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*, 2021, pp. 30–39.
- [30] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang *et al.*, “Codexglue: A machine learning benchmark dataset for code understanding and generation,” *arXiv preprint arXiv:2102.04664*, 2021.
- [31] M. A. Ferrag, A. Battah, N. Tihanyi, R. Jain, D. Maimuř, F. Alwahedi, T. Lestable, N. S. Thandi, A. Mechri, M. Debbah *et al.*, “Securefalcon: Are we there yet in automated software vulnerability detection with llms?” *IEEE Transactions on Software Engineering*, 2025.
- [32] M. T. Alam, R. Halder, and A. Maiti, “Detection made easy: Potentials of large language models for solidity vulnerabilities,” *arXiv preprint arXiv:2409.10574*, 2024.
- [33] N. I. of Standards and T. (NIST), “National vulnerability database (nvd) dashboard,” <https://nvd.nist.gov/general/nvd-dashboard>, 2025, accessed: 2025-04-24.
- [34] J. D. Pereira, J. H. Antunes, and M. Vieira, “A software vulnerability dataset of large open source c/c++ projects,” in *2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing (PRDC)*. IEEE, 2022, pp. 152–163.
- [35] MITRE Corporation, “CWE-1000: Research Concepts,” <https://cwe.mitre.org/data/definitions/1000.html>, 2024, accessed: 2025-10-18.
- [36] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, “Imagenet large scale visual recognition challenge,” *International journal of computer vision*, vol. 115, pp. 211–252, 2015.
- [37] N. Craswell, “Mean reciprocal rank,” *Encyclopedia of database systems*, pp. 1703–1703, 2009.
- [38] S. S. Shapiro and M. B. Wilk, “An analysis of variance test for normality (complete samples),” *Biometrika*, vol. 52, no. 3-4, pp. 591–611, 1965.
- [39] R. A. Fisher, “Statistical methods for research workers,” in *Breakthroughs in statistics: Methodology and distribution*. Springer, 1970, pp. 66–70.
- [40] H. B. Mann and D. R. Whitney, “On a test of whether one of two random variables is stochastically larger than the other,” *The annals of mathematical statistics*, pp. 50–60, 1947.
- [41] D. S. Kerby, “The simple difference formula: An approach to teaching nonparametric correlation,” *Comprehensive Psychology*, vol. 3, pp. 11–17, 2014.
- [42] J. Cohen, *Statistical power analysis for the behavioral sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.

- [43] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [44] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei *et al.*, “StarCoder 2 and the stack v2: The next generation,” *arXiv preprint arXiv:2402.19173*, 2024.
- [45] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [46] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, “Mistral 7b,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [47] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [48] OpenAI, “Gpt-4.1-mini,” <https://openai.com/index/gpt-4-1>, 2025, large language model by OpenAI. Accessed via ChatGPT or the OpenAI API.
- [49] V. Xiang, C. Snell, K. Gandhi, A. Albalak, A. Singh, C. Blagden, D. Phung, R. Rafailov, N. Lile, D. Mahan *et al.*, “Towards system 2 reasoning in llms: Learning how to think with meta chain-of-thought,” *arXiv preprint arXiv:2501.04682*, 2025.
- [50] A. Zibacirad and M. Vieira, “Reasoning with llms for zero-shot vulnerability detection,” *arXiv preprint arXiv:2503.17885*, 2025.

APPENDIX: DECODING TEMPERATURE AND OUTPUT VALIDITY (ADDED)

We audit raw SVD responses from deterministic decoding ($T=0$), two independent $T=0.1$ runs, and $T=1.0$. A genuine abstention clearly selects `not sure`, including a verbose response only when its final intent is unambiguously uncertainty due to insufficient information. Invalid responses include code continuation, malformed or looping text, contradictory choices, and outputs with no resolvable verdict. Table XIX reports both rates as percentages of all responses, pooled over non-targeted/targeted settings and PBD/LFD; the $T=0.1$ column is the arithmetic mean of the two runs.

TABLE XIX
GENUINE ABSTENTION AND INVALID-RESPONSE RATES (% OF ALL RESPONSES), SHOWN AS GENUINE/INVALID. RATES ARE POOLED OVER BOTH SVD SETTINGS AND BOTH DATASETS; $T=0.1$ IS AVERAGED ACROSS TWO RUNS. GPT-4.1-MINI WAS AVAILABLE ONLY AT $T=0.1$.

Model	T=0	T=0.1 (two-run avg.)	T=1.0
CodeLlama	0.00/1.80	0.00/1.83	2.34/1.20
DeepSeek-R1	0.00/0.00	0.00/0.00	0.00/0.00
GPT-4.1-mini	–	2.40/0.00	–
Llama3.1	0.00/0.00	0.00/0.00	0.00/0.00
Mistral	0.00/1.38	0.00/1.47	0.48/0.18
Qwen3-Coder	0.06/0.48	0.12/0.57	1.80/0.18
StarCoder2	0.00/26.08	0.00/22.45	1.14/1.92

The corrected audit does not support a universal 0% abstention claim. Genuine abstention is still uncommon but averages 2.40% for GPT-4.1-mini at $T=0.1$, reaches 2.34% for CodeLlama at $T=1.0$, and reaches 1.80% for Qwen3-Coder at $T=1.0$. Invalid output is a separate and often larger failure mode. It is concentrated in StarCoder2 (26.08% at $T=0$ and 22.45% averaged across the two $T=0.1$ runs), then drops to 1.92% at $T=1.0$. Thus, most StarCoder2 non-answers reflect prompt-format non-compliance rather than epistemic uncertainty, whereas GPT-4.1-mini’s observed non-binary responses are genuine abstentions.